

TDV 2215 Display IO Analysis - Missing Updates During Configuration Editing

Problem Analysis

You can change configuration values with arrow keys, but UI updates only appear after exiting with 0xED. The "Transmission speed" menu shows garbled text, indicating display update issues during editing mode.

Display Hardware Architecture

TMS9937/CRT5027 Controller IO Ports (0x30-0x3F)

- **Port 0x30-0x3F:** CRT controller registers
- **Port 0x60:** Status/Control port (key IO operations)
- **Port 0x10:** Hardware status port

Key Display Control Variables

Variable	Address	Purpose
AttributeRegisterValue	0x5df6	Current display attribute
display_refresh_control	0x5df9	Display refresh state
value_for_statusport_2	0x5e0e	Output value for port 0x60
statusport2_flag	0x5e0f	Control flag for port updates

Critical Display Update Functions

1. FUN_ram_1596(parameter) - Attribute Setting

```
c
byte FUN_ram_1596(byte attribute) {
    AttributeRegisterValue = attribute;
    disableMaskableInterrupts();
    value_for_statusport_2 = value_for_statusport_2 | 1; // Set bit 0
    enableMaskableInterrupts();
    return AttributeRegisterValue;
}
```

Purpose: Sets display attributes (normal=0, highlight=3, etc.) **IO Effect:** Sets bit 0 of port 0x60 output value

2. FUN_ram_1650() - Character Output with Refresh

c

```
undefined1 FUN_ram_1650(char character) {  
    *video_memory = character; // Write to video memory  
    while ((readInterruptMask() & 0x80) != 0); // Wait for hardware ready  
    display_refresh_control = display_refresh_control | 1; // Trigger refresh  
    return character;  
}
```

Purpose: Outputs character and triggers display refresh **IO Effect:** Sets refresh control flag

3. write_AttributeRegister_And_ControlPort2() - Hardware Update

c

```
byte write_AttributeRegister_And_ControlPort2() {  
    disableMaskableInterrupts();  
    statusport2_flag = 0; // Enable port output  
    enableMaskableInterrupts();  
    return value_for_statusport_2; // Output to port 0x60  
}
```

Purpose: Actually outputs control data to hardware **IO Effect:** Outputs value_for_statusport_2 to port 0x60

4. VSYNC Interrupt Handler - Display Refresh

c

```
// In vsync_interrupt_handler()  
display_refresh_control = (display_refresh_control & 3) << 1;  
if (statusport2_flag == 0) {  
    output_port_0x60 = value_for_statusport_2; // ACTUAL IO OUTPUT  
}
```

Purpose: Handles display refresh during VSYNC **IO Effect:** Outputs to port 0x60 during vertical blanking

The Bug: Missing Display Updates During Editing

In Initial Option Display (FUN_ram_1aff() start)

c

```
do {
    bVar2 = bVar2 + 1;
    FUN_ram_1c23();           // Position cursor
    uVar1 = 0;
    if (bVar4 == bVar2) {
        uVar1 = 3;           // Highlight current option
    }
    FUN_ram_1596(uVar1);      // ✓ SET ATTRIBUTE
    FUN_ram_1bdc();           // ✓ RENDER OPTION TEXT
} while (BYTE_ram_5e39 != bVar2);
```

This works: Each option is displayed with proper attributes.

In Arrow Key Navigation Loop (BROKEN)

```
c

do {
    bVar2 = get_keyboard_char_atomic();
    if (bVar2 == 0xb4) { // LEFT ARROW
        uVar3 = (ushort)(byte)(bVar4 - 1);
        goto LAB_ram_1b2d; // ✗ ONLY calls FUN_ram_1c23()
    }
    if (bVar2 == 0xb3) { // RIGHT ARROW
        uVar3 = (ushort)(byte)(bVar4 + 1);
        goto LAB_ram_1b2d; // ✗ ONLY calls FUN_ram_1c23()
    }
} while(true);

LAB_ram_1b2d:
    FUN_ram_1c23(); // ✗ ONLY POSITIONS CURSOR - NO ATTRIBUTE UPDATE!
```

The Problem: When you press arrow keys, the code:

- ✓ Updates the selection value (uVar3)
- ✓ Positions the cursor (FUN_ram_1c23())
- ✗ **NEVER** calls (FUN_ram_1596()) to update attributes
- ✗ **NEVER** calls (FUN_ram_1bdc()) to re-render options

Missing IO Operations During Editing

What Should Happen on Arrow Key Press

- Update Selection:** Change current option index ✓

2. **Set New Attribute:** Call `FUN_ram_1596(3)` for highlight ❌
3. **Clear Old Attribute:** Call `FUN_ram_1596(0)` for previous option ❌
4. **Re-render Options:** Call `FUN_ram_1bdc()` to display changes ❌
5. **Update Hardware:** Trigger `write_AttributeRegister_And_ControlPort2()` ❌

Current Behavior

- Selection changes internally ✅
- Cursor position updates ✅
- No visual feedback until exit ❌
- No attribute register updates ❌
- No port 0x60 outputs ❌

The "Transmission Speed" Garbled Text Issue

The garbled text for "Transmission speed" suggests:

1. **Configuration template loading works** (text appears)
2. **Display positioning fails** (wrong location/attributes)
3. **Character encoding issues** (garbled appearance)
4. **Attribute register corruption** (wrong display mode)

This happens because `FUN_ram_1f2d()` processes configuration templates but the attribute state is inconsistent.

Required Emulator Fixes

1. Port 0x60 Output Monitoring

Your emulator needs to properly handle:

```
c
// During VSYNC interrupt
if (statusport2_flag == 0) {
    write_to_port_0x60(value_for_statusport_2);
}
```

2. Attribute Register State Tracking

Monitor `AttributeRegisterValue` changes:

- 0: Normal text

- **3**: Highlighted text (current selection)
- **0xE**: Special attribute mode

3. Display Refresh Control

Track `display_refresh_control` flag:

- When set, trigger screen update during next VSYNC
- Clear after processing

4. Video Memory vs Attribute Separation

The TMS9937/CRT5027 likely has:

- **Character memory**: At video RAM addresses
- **Attribute memory**: Separate attribute plane
- **Control registers**: Via ports 0x30-0x3F

Debugging Steps

1. Monitor Key Variables During Editing

- `AttributeRegisterValue` (should change 0→3→0 when cycling)
- `value_for_statusport_2` (should have bit 0 set when attributes change)
- `display_refresh_control` (should trigger on updates)

2. Trace Function Calls During Arrow Keys

When pressing LEFT/RIGHT arrows, you should see:

-  Currently: Only `FUN_ram_1c23()`
-  Should see: `FUN_ram_1596()` + `FUN_ram_1bdc()` + refresh triggers

3. Fix The Missing Display Updates

The firmware has a bug where arrow key navigation doesn't trigger display updates. The fix would be to add the missing calls in the arrow key handling:

c

```
// What the code SHOULD do on arrow keys:
if (bVar2 == 0xb3 || bVar2 == 0xb4) {
    // Update selection
    uVar3 = (ushort)(byte)(bVar4 ± 1);

    // MISSING: Re-render all options with new highlighting
    for (each_option) {
        attribute = (current_option == new_selection) ? 3 : 0;
        FUN_ram_1596(attribute);
        FUN_ram_1bdc();
    }
    goto LAB_ram_1b2d;
}
```

Conclusion

The issue is **firmware-level**: the arrow key navigation code fails to trigger the display attribute and rendering updates that happen during initial option display. Your emulator needs to properly handle port 0x60 outputs and attribute register changes, but the fundamental problem is that the firmware doesn't call the right display functions during editing navigation.

The "Transmission speed" garbled text confirms that the display system partially works but has attribute/positioning issues, likely due to the same missing update calls.